Лабораторная работа 1

Приём работ и расширяемые анализаторы

Эммануэль Камате
Группа: M80-103CB-26
Дисциплина: «Библиотеки и фреймворки Python»
Дата отчёта: 14 октября 2026 г.

Цель и выбранная траектория

Реализована учебная основа системы приёма лабораторных работ. Выбрана траектория Python Core, основной уровень: dataclass, строгая проверка входа, Protocol, композиция, фабрики анализаторов, источники и тесты. Внешние entry points и декоратор измерения времени как углубление не реализованы. В заключительной рубрике задания основной уровень допускает максимальную оценку; противоречащая ему формулировка об обязательном углублении отмечена как неоднозначность задания.

Итог реализации

Создан устанавливаемый пакет school-lab в src-layout. Команда импорта возвращает итоговый JSON; отдельная команда анализа сохраняет отчёты JSONL. Имеются два источника строк и три анализатора: length, template, ai_declaration. Полученный код не исполняется, HTTP не используется.

Проверенный результат

В новой среде Linux / Python 3.12.14 установлено обычное wheel-распространение без editable-режима. Пройдены 35 тестовых методов. Для 12 синтетических работ получено 36 отчётов без ошибок. Студент ранее подтвердил 26 тестов на Windows / Python 3.14.7. Итоговую изменённую версию необходимо повторно проверить на Windows перед сдачей.

1. Приём и модель данных

Границы ответственности

Submission и AIDeclaration представлены frozen dataclass. Парсер получает данные и возвращает Submission либо SubmissionError(code, field, message). Он не читает stdin, не выполняет сетевые запросы и не изменяет вход. CLI отвечает за аргументы и файлы; репозиторий отвечает за хранение; сервис отвечает за duplicate/conflict.

Проверки входа

Отклоняются повторяющиеся JSON-ключи, NaN и бесконечности, повреждённый JSON, неизвестные и отсутствующие поля. Требуется schema_version=1 с точным типом int, исключающим bool. Идентификаторы непусты; timestamp включает зону; text или code содержит непустой текст. В декларации поля used и verified_by_student имеют тип bool; при used=true требуются непустые tool и purpose. Аннотации описывают контракт, но не выполняют эти проверки.

Устойчивое представление

Все поля Submission преобразуются через asdict в канонический JSON: ключи сортируются, разделители компактны, Unicode сохраняется, NaN запрещён. UTF-8 байты передаются SHA-256; результат имеет 64 шестнадцатеричных символа. Пробелы внутри текста и кода не удаляются. Строка timestamp сохраняется: эквивалентные моменты в разных строковых формах могут иметь разные хеши. Встроенный hash() не используется как устойчивый идентификатор содержимого.

Приём повторов

Новый submission_id сохраняется. Повтор того же ID с тем же хешем возвращает ранее сохранённый объект. При отличающемся содержимом возникает conflict без замены. Новая версия использует новый ID. Хранилище in-memory прекращает существовать после завершения процесса; операция get/add не является конкурентной транзакцией.

2. Анализаторы и контракты

Интерфейс и композиция

Analyzer(Protocol) задаёт синхронный метод analyze(submission, context) → AnalysisReport. Явное наследование от Protocol не требуется: существенны доступные операции и их типы. В отличие от этого, наследование SubmissionError от Exception определяет семейство исключений. Контекст содержит Assignment и снимок дополнительных работ. Протокол источника определяет open(); реализации выдают строки через контекстный менеджер.

Три реализации

LengthAnalyzer считает непустые строки text/code, включая комментарии, выдаёт rubric_feedback и score=null. TemplateAnalyzer ищет полный точный непрерывный блок разрешённого шаблона, сохраняет отступы и поясняет разрешённое совпадение; это не доказательство плагиата или оригинальности. AIDeclarationAnalyzer описывает декларацию, проверяет обязательные сведения при used=true и требует просмотр при неподтверждённой проверке студентом. Декларация не является доказательством авторства или правильности.

Расширяемость

Реестр сопоставляет имя с фабрикой. Центральный сервис создаёт выбранный объект и вызывает общую операцию analyze; ветвления по конкретным классам нет. Третий анализатор подключён новой записью реестра, без изменения алгоритма выбора. Новый экземпляр создаётся для каждого вызова.

Результаты и защита

Отчёт содержит ID и хеш работы, ID и версию анализатора, config_hash, observations/findings, время с зоной и review_required. Evidence ссылается на исходные строки с нумерацией от 1; границы включены. Сервис передаёт копии входа, выявляет сохраняющиеся изменения и проверяет метаданные и границы отчёта. Неожиданное исключение компонента преобразуется в analyzer_error; некорректный отчёт отклоняется. Это защита от ошибок компонента, а не песочница для вредоносного кода.

3. Источники, проверки и измерения

Жизненный цикл ресурса

InMemoryLineSource сохраняет строки в tuple и при открытии создаёт новый итератор. JsonlFileSource открывает UTF-8 файл и закрывает его при выходе из with. Общий контракт проверяет порядок, Unicode и повторное открытие. Отдельно проверены закрытие после чтения одной строки и закрытие при исключении. Сам по себе break не гарантирует закрытие произвольного итератора; ресурс освобождается при выходе из управляющего with.

Покрытые сценарии

35 методов проверяют корректный приём, неизвестные/отсутствующие поля, неверные типы и версию, дату без зоны, пустой контент, декларацию ИИ, повреждённый JSON, duplicate, conflict, неизменность входа, стабильность хеша, общие контракты трёх анализаторов, конфигурацию, evidence, отказ анализатора, ресурсы, каталог заданий и команды в отдельных процессах. Исчерпывающее доказательство корректности этим не утверждается.

Проверочные запуски

submissions.jsonl: accepted=12, duplicate=0, errors=0. submissions_demo.jsonl: accepted=2, duplicate=1, errors=2. invalid_submissions.jsonl: accepted=0, duplicate=0, errors=4. Анализ публичного корпуса тремя анализаторами: 36 отчётов. Коды завершения: 0 — успех, 1 — ошибки отдельных записей/анализаторов, 2 — аргументы или файлы. Диагностика импорта идёт в stderr; итоговый JSON — в stdout.

Небольшое измерение

В Linux / Python 3.12.14 выполнено 5 последовательных повторов анализа 12 работ тремя анализаторами (36 отчётов за повтор). Медиана: 5.394 мс. Значения: 5.394, 3.651, 6.365, 6.423, 4.722 мс. Измерен только вызов анализаторов через сервис: загрузка исходных файлов и запуск интерпретатора исключены. Включены копирование, SHA-256, проверка отчётов и генерация метаданных. Это измерение малого учебного корпуса, не сравнение производительности и не прогноз production-нагрузки. Полные значения: MEASUREMENTS.json; журнал: VERIFICATION.txt.

4. Воспроизведение и защита

Порядок запуска

1. Распаковать проект. 2. Создать .venv: python -m venv .venv. 3. Установить: python -m pip install . с интерпретатором среды. 4. Запустить: python -m unittest discover -v. 5. Импортировать submissions.jsonl и submissions_demo.jsonl. 6. Запустить school_lab.analyze_cli с --assignments assignments.json, выбранными --analyzer и новым --output. Точные команды PowerShell приведены в README.md.

Происхождение данных

12 публичных синтетических записей извлечены из PUBLIC_SUBMISSIONS в преподавательском notebook «Лекция.ipynb» без исполнения остальных ячеек. invalid_submissions.jsonl, assignments.json и template.txt подготовлены как явные демонстрационные входы проекта. Встроенные требования каталога не являются автоматической оценочной рубрикой: текущие анализаторы не оценивают семантическое выполнение требований.

Ограничения

Нет постоянной БД, HTTP API, многопоточности, автоматической оценки решений или детектора ИИ. Источник памяти материализует строки. Контекст дополнительных работ пока не заполняется CLI. При ошибке записи файл отчётов может быть частичным. Дубликат в одном запуске не анализируется повторно, даже если предыдущая попытка анализа дала ошибку. Повторный запуск требует нового имени выходного файла. Конфигурация измеряется и идентифицируется, но полная система аудита изменений не реализована.

Прозрачность и завершение сдачи

ChatGPT использовался для объяснений, разработки кода, тестов и документации; объём помощи отражён в AI_USAGE.md. Студент должен перечитать окончательный код и повторить проверку на Windows. Итоговый Git-тег lab-1 создаётся в репозитории студента после проверки; в этой архивной поставке тег не создан. Исторический stage-a не фабрикуется. Файлы README.md, CHANGES_FR.md и исходники помогают подготовить устное объяснение.

Источники требований

«ЛР_1_Приём_и_расширяемость.pdf», разделы «Этап А», «Этап Б», «Обязательные проверки», «Сдача и оценивание»; «Лекция.ipynb», общий контракт, примеры источников и анализаторов, контрольный список сдачи.